МИНСПОРТ РОССИИ

ФСП федерация спортивного программирования России

КУБОК ФЕДЕРАЦИИ 2026

РАЙТАП · CRYPTO

ВСЁ ПО ГОСТУ

Всё по ГОСТу

CRYPTO

19 СЕНТЯБРЯ 2026
ОНЛАЙН · CTF TASK-BASED

CRYPTO
КАТЕГОРИЯ

21
СЛАЙДОВ В РАЗБОРЕ

01

ИСХОДНЫЕ ДАННЫЕ

Категория: crypto. Всё, что ниже, выводится **только** из того, что видит игрок: раздача (`intercepted.txt`, `encrypt.py`) и сетевой сервис подписи. Ни строчки серверного кода мы не читаем — всё восстанавливается по открытым данным и поведению.

ВСЁ ПО ГОСТУ
CRYPTO

02

РАЗВЕДКА

Из описания: служебную переписку перевели на «отечественную криптографию, чтобы всё было по ГОСТу»; перехвачено сообщение, зашифрованное на открытый ключ получателя (без его закрытого ключа не читается); а тот же ключ обслуживает публичный сервис подписи, который «судя по всему, **реализован не идеально**». Последняя фраза — прямая наводка: изъян искать в реализации подписи. В раздаче лежат `intercepted.txt` и `encrypt.py`.

Смотрим конверт:

```
$ cat intercepted.txt
Ex  = 209bb2c6ab11292a9afc1a45ffbcb316...
Ey  = b7593e2f96034161d958d1822f831df0...
ukm = 3226bad257705280
iv  = 635d01e3b322b358
ct  = 488700da54473eea22af09885291c26a...
```

Голый блоб: две координаты какой-то точки `E`, короткие `ukm` / `iv` и шифртекст `ct`. Пахнет гибридным шифрованием на эллиптике (ECIES): `E` — эфемерный публичный ключ отправителя, `ct` — тело, зашифрованное симметрично.

Подключаемся к сервису:

02

РАЗВЕДКА

ПРОДОЛЖЕНИЕ

```
$ nc <HOST> <PORT>
Сервис электронной подписи. Команды: PUBKEY | SIGN [msg] | HELP | QUIT
PUBKEY
curve=A
p=ffffff
```

02

РАЗВЕДКА

ПРОДОЛЖЕНИЕ

Итого: параметры кривой + открытый ключ `Y`, подпись `(r, s)` с хэшем `e`, и — самое интересное — **две метки времени**. Тайминг в крипто-сервисе почти всегда сайд-канал; отложим в голове и пойдём опознавать саму систему.

Кстати, про «по ГОСТу» — это не только имя таска, но и честный анонс: дальше по стеку нас ждёт ГОСТ на подпись, ГОСТ на согласование ключа и ГОСТ на шифр, отечественное импортозамещение во всей красе. И маленькая ирония напоследок: в настоящем LadderLeak утечку получают из кэша процессора, борясь с джиттером сети и планировщика. Здесь же используется детерминированная симуляция: сервис сам печатает две метки, разность которых точно равна длине nonce. Это учебная модель утечки bit-length nonce, а не воспроизведение LadderLeak.

Тут же начнём **собирать солвер** — по кирпичику, ровно по ходу рассуждений. Первый кирпич самый скучный: коннект, пропускаем баннер, забираем публичные параметры.

ВСЁ ПО ГОСТУ
CRYPTO

02

РАЗВЕДКА

ПРОДОЛЖЕНИЕ

```python
def _connect():
    s = socket.create_connection((HOST, PORT))
    f = s.makefile("rwb")
    f.readline()                                  # баннер пропускаем
    return s, f

def get_pubkey(f):
    f.write(b"PUBKEY\n"); f.flush()
    d = {}
    for line in _read_block(f):
        if "=" in line:
            k, v = line.split("=", 1); d[k.strip()] = v.strip()
    p = int(d["p"], 16); a = int(d["a"], 16); q = int(d["q"], 16)
    G = (int(d["Gx"], 16), int(d["Gy"], 16))
    Y = (int(d["Yx"], 16), int(d["Yy"], 16))
    return p, a, q, G, Y
```

ВСЁ ПО ГОСТУ
CRYPTO

03

ЧТО ЗА КРИПТОСИСТЕМА (ПО ОТКРЫТЫМ ДАННЫМ)

Описание про стандарт молчит, но константы кривой говорящие. `p` и `q` вида `FFFF…FDC7` / `FFFF…0B275` — это published-параметры `id-tc26-gost-3410-12-512-paramSetA` (Р 1323565.1.024-2019, они же в RFC 7836). То есть подпись — **ГОСТ Р 34.10-2012**, 512 бит.

Немного контекста, раз уж «всё по ГОСТу». 34.10-2012 — российский стандарт ЭП на эллиптических кривых, принят в 2012-м на смену 34.10-2001, ведёт его технический комитет ТК-26. Рядом ходит вся отечественная линейка: хэш «Стрибог» (34.11-2012, назван в честь славянского бога ветра) и блочный шифр «Кузнечик» (34.12-2015). У «Кузнечика», к слову, была громкая история: в 2016-м Бирюков, Перрен и Удовенко раскопали в его S-box'е скрытую алгебраическую структуру, которую авторы стандарта подавали как «случайную». На стойкость это (пока) не повлияло, но осадочек и хорошая байка остались. Нам всё это на руку — стек предсказуемый.

Проверяем гипотезу честно — уравнением верификации (секрет не нужен, всё по `Y`). Для ГОСТ Р 34.10-2012 проверка такая: `v = e^{-1} mod q`, `z1 = s·v`, `z2 = (q - r)·v`, `C = z1·G + z2·Y`, и должно быть `C.x mod q == r`. Берём любую пару из `SIGN` и считаем — сходится. Значит это точно ГОСТ 34.10 (у ECDSA уравнение другое, и `R ≠ r`).

Почему проверка вообще срабатывает без секрета? Подставим `s = r·d + k·e`:

```
z1·G + z2·Y = s·v·G - r·v·(d·G) = v·(s - r·d)·G = v·(k·e)·G = k·G
```

ВСЁ ПО ГОСТУ
CRYPTO

03

ЧТО ЗА КРИПТОСИСТЕМА (ПО ОТКРЫТЫМ ДАННЫМ)

ПРОДОЛЖЕНИЕ

`C` схлопывается ровно в `k·G` — ту самую точку, чей `x mod q` и даёт `r`. Изящно: уравнение верификации втихую пересчитывает `k·G`, не зная ни `k`, ни `d`. Этот же трюк — «прогнать публичные данные через чужое уравнение и посмотреть, сойдётся ли» — вообще лучший способ вслепую опознать схему подписи.

Теперь `encrypt.py` — это открытая схема, которой зашифровали письмо:

```
# encrypt.py (выдержка)
def vko512(scalar, Pt, ukm, a, p, q):
    K = multiply_point((ukm * scalar) % q, Pt, a, p)          # общая точка
    return streebog512(K[0].to_bytes(fb,"little") + K[1].to_bytes(fb,"little"))

def ecies_encrypt(plaintext, p, a, q, G, Y):
    r_e = secrets.randbelow(q-2)+1
    E   = multiply_point(r_e, G, a, p)
    KEK = vko512(r_e, Y, ukm, a, p, q)                        # K = (ukm·r_e)·Y
    ct  = kuznechik_ctr(KEK, iv, plaintext)                   # Кузнечик-CTR
    return E, ukm, iv, ct
```

Читаем: это ECIES по-госту — согласование ключа **VKO_GOSTR3410_2012_512** (RFC 7836), контент шифруется **Кузнечиком** (ГОСТ 34.12-2015) в CTR. Ключевой момент — общий ключ:

```
отправитель:  KEK = Streebog512( (ukm·r_e)·Y )
получатель:   KEK = Streebog512( (ukm·d )·E )
              (ukm·r_e)·Y = (ukm·r_e·d)·G = (ukm·d)·E   → та же точка
```

ВСЁ ПО ГОСТУ
CRYPTO

03

ЧТО ЗА КРИПТОСИСТЕМА (ПО ОТКРЫТЫМ ДАННЫМ)

ПРОДОЛЖЕНИЕ

Вывод: чтобы прочитать письмо, нужен **закрытый ключ получателя** `d`. А тот же `d` обслуживает сервис подписи. Значит цель — вытащить `d` из подписей.

И сразу снимем соблазн искать «дыру в шифраторе». То, что нам выдали сам `encrypt.py`, — не подарок и не баг: по принципу Керкгоффса (сформулирован ещё в 1883-м) стойкость обязана держаться на секретности ключа, а не алгоритма. Открытый шифратор не расшифровывает ни на грамм — он умеет только шифровать на публичный `Y` и `d` в глаза не видел. Так что путь один: добывать `d`.

ВСЁ ПО ГОСТУ
CRYPTO

04

ПОБОЧНЫЙ КАНАЛ: ТАЙМИНГ ВЫДАЁТ НОНС

Возвращаемся к меткам времени. Снимем разницу `Δt = t_готово - t_начал` по многим подписям:

```
import socket, re
TS = re.compile(r"\[(\d\d):(\d\d):(\d\d)\.(\d+)\]")
# ... коннектимся, шлём SIGN N раз, парсим две метки, копим Δt в мс ...
```

Картина: почти все `Δt` кучкуются у некоторого максимума, но изредка попадаются заметно **меньшие**. Что подписывается на кривой? Скалярное умножение `[k]G`, где `k` — одноразовый секретный нонс. Время такого умножения пропорционально **числу шагов**, то есть **битовой длине** нонса. Меньше `Δt` ⇒ короче нонс ⇒ у него **нулевые старшие биты**.

Это детерминированная симуляция утечки **bit-length nonce**. Её последствия для решения совпадают с классом атак на частично известные nonce: для коротких nonce известны ведущие нули, а затем применяется HNP и решётка. LadderLeak — близкий по тематике, но другой по модели источник:

> D. F. Aranha, F. R. Novaes, A. Takahashi, M. Tibouchi, Y. Yarom. *«LadderLeak: Breaking ECDSA With Less Than One Bit Of Nonce Leakage»*, ACM CCS 2020. Утечка времени в реализации лестницы раскрывает старший бит нонса, и этого хватает для восстановления ключа. Референс-реализация: https://github.com/akiratk0355/ladderleak-attack-ecdsa

ВСЁ ПО ГОСТУ
CRYPTO

04

ПОБОЧНЫЙ КАНАЛ: ТАЙМИНГ ВЫДАЁТ НОНС

ПРОДОЛЖЕНИЕ

Небольшое отступление — провалы с нонсом это вообще отдельный жанр, и он богат. Самый хрестоматийный экспонат — Sony PS3 (2010): прошивки подписывали ECDSA с **одним и тем же** `k` на всё, и ребята из fail0verflow достали приватный ключ буквально из двух автографов. Потом пошли утечки поизящнее: **Minerva** (2019) и **TPM-FAIL** (2019) — там время подписи зависело от длины нонса в популярных библиотеках и прямо в TPM-чипах, то есть использовали тот же класс утечки длины nonce. **LadderLeak** довёл жанр до абсурда красоты: научились вынимать ключ, зная **меньше одного бита** нонса на подпись, докручивая статистику Бляйхенбахером через FFT. Нам, честно говоря, повезло больше — утечка чистая и не по крохе бита, так что тяжёлую артиллерию не достаём, обойдёмся решёткой.

Калибруемся вслепую: делим `Δt` на известную разрядность `w = bitlen(q) = 512` — получаем «цену одного шага», а по ней для каждой подписи оцениваем `t = bitlen(k)` и число ведущих нулей `lz = w - t`. Отбираем подписи с `lz ≥ L` (у них старшие `L` бит нонса гарантированно нули). Такие попадаются примерно раз в `2^L` подписей — это цена за один пригодный образец.

Следующий кусок солвера. `sign_once` шлёт `SIGN`, вытаскивает две метки и отдаёт подпись плюс задержку в миллисекундах:

04

ПОБОЧНЫЙ КАНАЛ: ТАЙМИНГ ВЫДАЁТ НОНС

ПРОДОЛЖЕНИЕ

```python
def sign_once(f, msg=None):
    f.write((("SIGN " + msg) if msg else "SIGN").encode() + b"\n"); f.flush()
    stamps, kv = [], {}
    for line in _read_block(f):
        m = TS_RE.search(line)
        if m:                                       # метка времени в трейсе
            stamps.append(_ts_to_us(int(m.group(1)), int(m.group(2)),
                                    int(m.group(3)), m.group(4)))
        elif "=" in line:                           # r= / s= / e=
            k, v = line.split("=", 1); kv[k.strip()] = v.strip()
    elapsed_ms = (stamps[-1] - stamps[0]) / 1000.0
    return int(kv["r"],16), int(kv["s"],16), int(kv["e"],16), elapsed_ms
```

Квант нам не сообщают, но он вытаскивается из самих данных: длина нонса `t = round(elapsed/квант)`, а квант оцениваем по разбросу задержек и известному `w = bitlen(q)`:

```python
def calibrate_step(elapseds, w):
    step = sorted(elapseds)[int(len(elapseds)*0.97)] / w   # затравка ~ макс/w
    for _ in range(6):
        ts = [max(1, round(e/step)) for e in elapseds]
        step = statistics.median(e/t for e, t in zip(elapseds, ts))
    return step
```

Дальше на каждый образец считаем `lz = w - round(elapsed/step)` и оставляем только те, где `lz ≥ L` — вот и весь отбор «удачных» подписей.

05

ОТ ПОДПИСИ К HIDDEN NUMBER PROBLEM

У ГОСТ 34.10 компонента подписи: `s ≡ r·d + k·e (mod q)`. Выразим нонс через секрет:

```
k ≡ e^{-1}·s - e^{-1}·r·d      (mod q)
k ≡ A·d + U                    (mod q),   A = -e^{-1}·r,  U = e^{-1}·s
```

Мы знаем, что у отобранных подписей `k` «маленький» (`0 ≤ k < 2^{w-L}`), потому что старшие `L` бит — нули. То есть про каждую подпись известно приближённое линейное соотношение на неизвестный `d` по модулю `q` с малой ошибкой. Это в точности **задача поиска скрытого числа (HNP)**:

> D. Boneh, R. Venkatesan. *«Hardness of Computing the Most Significant Bits of Secret Keys in Diffie–Hellman and Related Schemes»*, CRYPTO 1996 — HNP сводится к поиску ближайшего вектора в решётке и решается редукцией базиса.
>
> P. Q. Nguyen, I. E. Shparlinski. *«The Insecurity of the Elliptic Curve Digital Signature Algorithm with Partially Known Nonces»*, DCC 30 (2003) — тот же приём на ECDSA: «знание нескольких старших бит нонса по набору подписей достаточно для восстановления секретного ключа».

Тут просится историческая ирония. Боне и Венкатесан придумали HNP в 1996-м, чтобы **доказать безопасность** — что старшие биты общего ключа Диффи–Хеллмана вычислить так же трудно, как весь ключ (результат «в плюс», в защиту). А спустя несколько лет Нгуен развернул ровно тот же инструмент в оружие против DSA/ECDSA; его работа так и называется — *«The Dark Side of the Hidden Number Problem»*. Тот же молоток, которым забивали гвоздь, оказался прекрасной фомкой. Мы сейчас на «тёмной стороне».

ВСЁ ПО ГОСТУ
CRYPTO

05

ОТ ПОДПИСИ К HIDDEN NUMBER PROBLEM

ПРОДОЛЖЕНИЕ

Интуиция, почему пары «старших нулей» хватает: каждая годная подпись — это узкая полоса, в которой обязан лежать `d` по модулю `q`. Одна полоса бесполезна, но десятки полос под разными углами пересекаются ровно в одной точке. Наша задача — геометрически поймать это пересечение, и вот тут выходит решётка.

Разница с ECDSA только в том, как из уравнения подписи получаются коэффициенты `A_i, U_i`. Для ГОСТа это выведено выше — дальше всё стандартно.

В коде это начало `recover_key`: на каждый образец — свои `A_i`, `U_i` и граница по его `lz` (чем больше ведущих нулей, тем точнее известен нонс, тем «весомее» строка):

```
for s in data:
    lz  = s["lz"]
    B   = 1 << (m - lz)              # нонс лежит в [0, B)
    half = B >> 1
    wsc = max(1, q // B)             # вес строки ~ 2^lz (балансировка)
    e_inv = pow(s["e"], -1, q)
    A_l.append((-e_inv * s["r"]) % q)      # A = -e^{-1}·r
    U = (e_inv * s["s"]) % q                # U = e^{-1}·s
    t_l.append((half - U) % q)              # центр интервала для d
    w_l.append(wsc)
```

ВСЁ ПО ГОСТУ
CRYPTO

06

РЕШЁТКА: ВОССТАНАВЛИВАЕМ ЗАКРЫТЫЙ КЛЮЧ

Собираем классическое вложение HNP (Boneh–Venkatesan / Nguyen–Shparlinski): по `m` подписям строим решётку размерности `m+1` с диагональю из `q` и строкой коэффициентов `A_i`, целевой вектор из центров интервалов `U_i`. Редуцируем базис (LLL/BKZ) и берём ближайший вектор — его последняя координата и есть `d`. В этой задаче решётчатое вложение реализовано напрямую в `attack_client.py` с коэффициентами `A_i, U_i` для ГОСТ.

Двигатель всего этого — алгоритм **LLL** (Ленстра, Ленстра, Ловас, 1982): за полиномиальное время он приводит базис решётки к «почти ортогональному», после чего короткий вектор виден невооружённым глазом. LLL — рабочая лошадь всей решёточной криптоаналитики: им ломали ранцевые шифры, малые экспоненты RSA (через Копперсмита) и вот такие нонс-атаки. Есть и предел: чем меньше бит утекает, тем «мутнее» задача для решётки, и на самом краю (1–2 бита) чистая решётка пасует — упирается в так называемый **решёточный барьер**. Тогда в ход идут predicate-BDD (Альбрехт–Хенингер, работа с говорящим названием *«Breaking the lattice barrier...»*) или всё тот же Бляйхенбахер. У нас `L = 8` — до барьера как до Луны, LLL справляется играючи.

Наш солвер (`attack_client.py`) делает всё сам: коннектится к сервису, многопоточно набирает подписи, из меток времени вычисляет `lz`, отбирает годные, строит решётку и восстанавливает `d`. Успех проверяется **только по открытому ключу** — `d·G == Y`:

Само сердце `recover_key` компактное — базис, ближайший вектор, сверка кандидата:

ВСЁ ПО ГОСТУ
CRYPTO

06

РЕШЁТКА: ВОССТАНАВЛИВАЕМ ЗАКРЫТЫЙ КЛЮЧ

ПРОДОЛЖЕНИЕ

```
basis = [[w_l[i]*A_l[i] for i in range(n)] + [1]]     # строка «множителя» d
for i in range(n):                                    # диагональ из q
    row = [0]*(n+1); row[i] = w_l[i]*q; basis.append(row)
target = [w_l[i]*t_l[i] for i in range(n)] + [0]

closest = _solve_cvp(basis, target)                   # редукция + Babai
d_cand  = closest[-1] % q                              # последняя координата = d
if fast_multiply(d_cand, G, a, p) == tuple(Y):         # d·G == Y ?
    return d_cand
```

`_solve_cvp` — это редукция базиса (LLL, при нужде BKZ) и ближайший вектор через `fpylll`; `fast_multiply` — умножение точки (в координатах Якоби, чисто ради скорости сверки). Многопоточный сбор и конвейер «сбор ‖ решётка» — уже обвязка, она в полном `attack_client.py`. Запускаем:

```
$ python3 attack_client.py 8
оракул      : <HOST>:<PORT>
кривая      : A   bitlen(q) = 512
leak L      : 8  (порог отбора; в решётку идёт фактический lz)
...
  оценён квант ≈ 0.0500 мс/шаг; сбор в 12 поток(ов)
  запрошено 24000, годных 87/100 ...
  попытка 1 (размер 67, 176 с): УСПЕХ
------------------------------------------------------------
[OK] d*G == Y — секретный ключ восстановлен.
  hex : 0x5f0e03555e31a4c2...508c6d257
```

ВСЁ ПО ГОСТУ
CRYPTO

06

РЕШЁТКА: ВОССТАНАВЛИВАЕМ ЗАКРЫТЫЙ КЛЮЧ

ПРОДОЛЖЕНИЕ

Почему именно `L = 8`? Тут размен «сбор ↔ решётка». Годная подпись попадается раз в `2^L`, поэтому **меньший `L` дешевле по сбору** (`L=6` — это ~8 тыс. запросов против ~25 тыс. при `L=8`), но решётка выходит **больше**: её размерность ≈ `w/L`, и она ближе к пределу качества редукции. Больший `L` — наоборот: решётка крохотная и берётся чистым LLL мгновенно, зато сбор растёт экспоненциально. Плоский LLL, к слову, тянет вплоть до `L≈4`; ниже уже нужны более сильные методы HNP. То есть BKZ — не про `L=6`, а про самый край.

Ключевой момент — **что здесь узкое место**. Сбор мы сделали быстрым (параллельные запросы, чистый тайминг), поэтому по стене упирается уже сама **решётка**, и тут выигрывает больший `L`: при `L=8` размерность всего ~68, и LLL щёлкает её мгновенно, тогда как `L=6` (размерность ~90) считается заметно дольше — несмотря на втрое меньший сбор. Отсюда и выбор восьмёрки: не потому что «меньше нельзя», а потому что на этом стенде так суммарно быстрее.

Ключ у нас. `d·G == Y` гарантирует, что это ровно тот `d`, что у получателя.

ВСЁ ПО ГОСТУ
CRYPTO

07

РАСШИФРОВКА ПИСЬМА И ФЛАГ

Теперь применяем `d` к конверту. Вся расшифровка держится на одном факте — **общий секрет Диффи–Хелл-мана симметричен**. `(ukm·r_e)·Y` и `(ukm·d)·E` — это буквально одна и та же точка, просто посчитанная с раз-ных концов: отправитель заходил со стороны эфемерного `r_e`, а мы заходим со стороны украденного `d`. В этом и весь смысл `E` в конверте — она наш «вход» в тот же секрет. Дальше «Кузнечик» (да, шифр так и зовётся — grasshopper) работает в CTR, а CTR — это по сути гамма: шифртекст = открытый текст XOR поток. Наложить тот же поток второй раз — и текст расшифрован, отдельная «функция расшифрования» не нужна:

```
# solve_flag.py
def vko512(scalar, Pt, ukm, a, p, q):                # VKO_GOSTR3410_2012_512
    fb = (p.bit_length()+7)//8
    K = multiply_point((ukm*scalar) % q, Pt, a, p)    # общая точка
    return streebog512(K[0].to_bytes(fb,"little") + K[1].to_bytes(fb,"little"))

def kuznechik_ctr(key, iv, data):                     # Кузнечик-CTR (симметричен)
    c = gostcipher.new("kuznechik", bytearray(key[:32]),
                       gostcipher.MODE_CTR, init_vect=bytearray(iv))
    return bytes(c.encrypt(bytearray(data)))

KEK = vko512(d, E, ukm, a, p, q)                      # тот же ключ, но со стороны d
pt  = kuznechik_ctr(KEK, iv, ct)                      # и вот оно, письмо
```

ВСЁ ПО ГОСТУ
CRYPTO

07

РАСШИФРОВКА ПИСЬМА И ФЛАГ

ПРОДОЛЖЕНИЕ

```
$ python3 solve_flag.py 0x5f0e03555e31a4c2...508c6d257 intercepted.txt
============================================================
РАСШИФРОВАННОЕ ПИСЬМО:
============================================================
СЛУЖЕБНАЯ ЗАПИСКА
...
Код активации: flag{p4v1l10n_r3b00t3d_1nt0_c0mput1ng}
...
```

Готово.

`flag{p4v1l10n_r3b00t3d_1nt0_c0mput1ng}`

ВСЁ ПО ГОСТУ
CRYPTO

08

СОЛВЕР ЦЕЛИКОМ, ЗАМЕТКИ И ИСТОЧНИКИ

Сложите куски по порядку — транспорт (`_connect` / `get_pubkey`) → замер и калибровка (`sign_once` / `calibrate_step`) → линеаризация в `A_i/U_i` → решётка с проверкой `d·G═Y` → расшифровка `vko512` + `kuznechik_ctr` — и получится ровно то, что лежит в `attack_client.py` (добыча ключа) и `solve_flag.py` (чтение письма). Весь наш полёт мысли материализовался в солвер: каждый абзац рассуждений — это его очередная функция.

Логика решения: опознать ГОСТ по открытым данным → заметить детерминированную утечку bit-length nonce → свести подпись к HNP → решёткой достать `d` → расшифровать ECIES (VKO_512 + Кузнечик). Единственный feasible путь к письму — восстановить `d`; «брутфорс» это ECDLP на 512-бит (~2^{256}), нерешаемо.

Источники:

- **LadderLeak** — Aranha, Novaes, Takahashi, Tibouchi, Yarom, ACM CCS 2020: источник для контекста, но не модель оракула в этой задаче. Репозиторий: https://github.com/akiratk0355/ladderleak-attack-ecdsa
- **HNP → решётки** — Boneh, Venkatesan, CRYPTO 1996.
- **Атака на ECDSA с частично известными нонсами** — Nguyen, Shparlinski, DCC 2003; Nguyen, *The Dark Side of the Hidden Number Problem: Lattice Attacks on DSA*, 2001.
- **Ниже одного бита / Бляйхенбахер** — De Mulder et al., CHES 2013.
- **ГОСТ Р 34.10-2012** (подпись), **RFC 7836 / VKO** (согласование ключа), **ГОСТ Р 34.12-2015 «Кузнечик»** (шифр контента).

ВСЁ ПО ГОСТУ
CRYPTO

ФЛАГ

```
flag{p4v1l10n_r3b00t3d_1nt0_c0mput1ng}
```

Задание «Всё по ГОСТу» решено